

Week 3 – Individual Assignment: Data Profiling and Cleaning

Submitted by: Shruti Bhandari

Programme: Master of Science in Data Science &
Artificial Intelligence

Institution: SRH University, Heidelberg

Tool Used: Tableau Prep Builder

Dataset: tableau_prep_cleaning_exercise.csv

1. Introduction

This report documents the complete data profiling and cleaning pipeline applied to a structured sales dataset using Tableau Prep Builder. The dataset contained 100 rows and 18 columns representing order transactions across multiple customers, countries, and product categories. The objective was to assess data quality, define cleaning rules, apply transformations, validate the results, and reflect on the appropriate pipeline context (ETL vs ELT) for this dataset.

2. Data Profiling (Pre-Cleaning)

Before any cleaning was performed, the dataset was profiled systematically in the **Clean 1 Profile Pane** of Tableau Prep Builder to identify all quality issues.

2.1 Missing Values

Eight out of 18 columns contained null values, totalling 76 null cells. The most critical were **Satisfaction_Score** (15%), **Payment_Method** (14%), and **Customer_Segment** (12%), as these fields are used directly in segmentation and analysis.

Column	Null Count	Null %	Cleaning Action
Satisfaction_Score	15	15%	Fill with median (3.0)
Payment_Method	14	14%	Fill with "Unknown"
Customer_Segment	12	12%	Fill with "Unknown"
Email	10	10%	Leave null
Notes	10	10%	Leave null
Age	7	7%	Fill with median (46.0)
Order_Date	5	5%	Leave null
City	3	3%	Leave null

In Tableau Prep, missing values are visible as a grey bar labelled *null* at the top of each column card in the Profile Pane.

2.2 Data Types

Three columns were stored with incorrect data types, preventing calculations, date filtering, and aggregation.

Column	Actual Type	Expected Type	Root Cause
Unit_Price	String (Abc)	Float/Decimal	Values contain € prefix and comma decimal separators
Order_Date	String (Abc)	Date	Six different date formats mixed in one column
Ship_Date	String (Abc)	Date	Mixed formats + text value "not available" in 2 rows

Six distinct date formats were found in Order_Date: YYYY/MM/DD, YYYY-MM-DD, DD/MM/YYYY, DD.MM.YYYY, MM-DD-YYYY, and DD-MM-YYYY.

2.3 Value Distributions

Key distribution findings are summarised below:

- **Quantity:** Ranges from 0 to 12. Four rows (O1007, O1035, O1056, O1070) had Quantity = 0, resulting in Sales = 0 — invalid transactions with no business value.
- **Age:** Ranges from 16 to 130. The value 130 (Order O1048) is impossible and a clear data entry error; all other valid ages fell between 16 and 76.
- **Sales:** Ranges from €0.00 to €2,604.13 (excluding sentinel-inflated rows). Fully consistent with the formula $\text{Quantity} \times \text{Unit_Price} \times (1 - \text{Discount})$.
- **Satisfaction_Score:** Values 1–5 only. Distribution: 1=9, 2=20, 3=22, 4=20, 5=14. Scale is valid.
- **Unit_Price:** 8 values had a € prefix, 5 had comma decimal separators, and 4 contained the sentinel placeholder value 9999.99.

2.4 Duplicates

Five exact duplicate row pairs were found, where every one of the 18 fields was identical:

Duplicate Order_ID	Sales Value
O1005	€782.64
O1018	€2,604.13
O1029	€1,828.86
O1048	€444.45
O1064	€1,371.33

As a result, Order_ID had only 95 unique values across 100 rows and could not yet serve as a primary key. The total revenue double-counted due to these duplicates was **€7,031.41**.

2.5 Cardinality of Key Columns

Column	Unique Values	Expected	Role	Issue
Order_ID	95	100	Primary key (intended)	5 duplicates
Customer_ID	46	Many	Foreign key	Normal — repeating by design
Country	15	5	Category	Case and language variants
Product_Category	10	4	Category	Case variants
Customer_Segment	6	3	Category	Case variants
Payment_Method	7	4	Category	Case variants
Email	15	Many	Contact field	Unusually low — shared emails

2.6 Inconsistent Text Formats and Invalid Patterns

Four category columns contained multiple case and language variants for the same real value:

- **Country:** 15 raw variants for 5 countries (e.g., Deutschland, germany, GERMANY, Germany → Germany)
- **Product_Category:** 10 raw variants for 4 categories (e.g., ELECTRONICS, electronics → Electronics)
- **Customer_Segment:** 6 variants for 3 segments (e.g., CORPORATE, corporate → Corporate)
- **Payment_Method:** 7 variants for 4 methods (e.g., PAYPAL, bank transfer → PayPal, Bank Transfer)
- **Ship_Date:** Two rows contained the literal string "not available" instead of a date value

3. Column Analysis

3.1 Columns with Many Unique Values

Columns were classified by their uniqueness ratio (unique values ÷ total rows):

Column	Unique Values	Uniqueness %	Classification
Order_ID	95	95%	Near-unique — intended primary key
Unit_Price	93	93%	Continuous measure
Sales	92	92%	Continuous measure
Ship_Date	87	87%	Temporal field
Order_Date	82	82%	Temporal field
Customer_ID	46	46%	Repeating foreign key
Age	43	43%	Numeric measure
Email	15	15%	Contact field — unexpectedly low
Country	15	15%	Category — inflated by case variants

After standardisation, Country, Product_Category, Customer_Segment, and Payment_Method reduce to their true unique counts of 5, 4, 3, and 4 respectively.

3.2 Identifier Candidates

Only **Order_ID** qualifies as a primary key — after removing the 5 duplicate rows, it will have 95 unique non-null values across 95 rows. Customer_ID is a foreign key that intentionally repeats, as one customer can place multiple orders. Email cannot serve as an identifier due to 10 nulls and only 15 unique values across 90 rows.

3.3 Date, Email, Category, and Numeric Validity

- **Order_Date:** 95 rows parsed successfully; 5 null; 9 future-dated orders found (flagged but not deleted as they represent scheduled orders).
- **Email:** All 90 present email addresses follow a valid name@domain.com format; 10 rows have null emails.
- **Categories:** All category values are semantically valid after case normalisation; no unknown or out-of-range category values exist.
- **Numeric columns:** Quantity=0 (4 rows), Age=130 (2 rows), and Unit_Price=9999.99 (4 rows) were the only invalid numeric values.

4. Structure Analysis

4.1 Order_ID as Record Identifier

Order_ID contains no nulls and is close to unique, making it the intended primary key. However, 5 Order_IDs each appear exactly twice as complete row duplicates. After removing one copy of each duplicate pair, Order_ID will have 95 unique values across 95 rows and can reliably serve as the primary key.

4.2 Customer_ID as Record Identifier

Customer_ID is a foreign key, not a primary key. It repeats because one customer can place multiple orders — Customer C019, for example, placed 5 orders. This is expected and correct behaviour by design. However, 31 out of 46 Customer_IDs were linked to more than one distinct customer name due to mixed-case entries (e.g., C054 linked to "CARLOS GARCIA", "JULIA FISCHER", and "Nina Weber"). After standardising Customer_Name to Title Case, most of these apparent mismatches resolve.

4.3 Product_ID vs. Product_Name Consistency

This dataset does not contain a Product_ID column — only a Product_Category column. The four real categories (Office Supplies, Electronics, Furniture, Home) are consistent in meaning; the only issue is casing inconsistency addressed during cleaning. No Product_ID to Product_Name referential integrity check was applicable.

4.4 Sales Field Consistency

The Sales column was verified against the formula: $\text{Calculated_Sales} = \text{Quantity} \times \text{Unit_Price} \times (1 - \text{Discount})$. Across all 100 rows, 100% matched exactly. The total difference of €0.01 was purely floating-point rounding and not a data error. Notably, three rows showed inflated Sales values (up to €56,999.94) caused entirely by the sentinel Unit_Price value of 9999.99 — corrected by replacing the sentinel with the median price.

5. Cleaning Rules

5.1 Missing Value Handling

Missing values were handled by assessing whether a column is analytical or informational:

- **Fill with median:** Satisfaction_Score (3.0) and Age (46.0) — median is robust to outliers.
- **Fill with "Unknown":** Payment_Method and Customer_Segment — preserves the row without fabricating a category.
- **Leave as null:** Email, Notes, City, and Order_Date — imputing these fields would introduce false data with no analytical benefit.

5.2 Duplicate Removal

All 5 exact duplicate pairs were removed — one copy of each was excluded. This reduced the row count from 100 to 95 and eliminated the €7,031.41 double-counted revenue.

5.3 Invalid Value Correction

Column	Invalid Value	Rule	Action
Age	130	Replace with median 46.0	IF [Age] > 100 THEN 46.0 ELSE [Age] END
Quantity	0	Exclude rows	Filter: [Quantity] > 0
Unit_Price	9999.99 (sentinel)	Replace with median 166.50	Calculated field with conditional
Ship_Date	"not available"	Replace with null	Edit Value in Tableau Prep

5.4 Cleaning Sequence

The cleaning steps must follow a strict sequence to avoid errors:

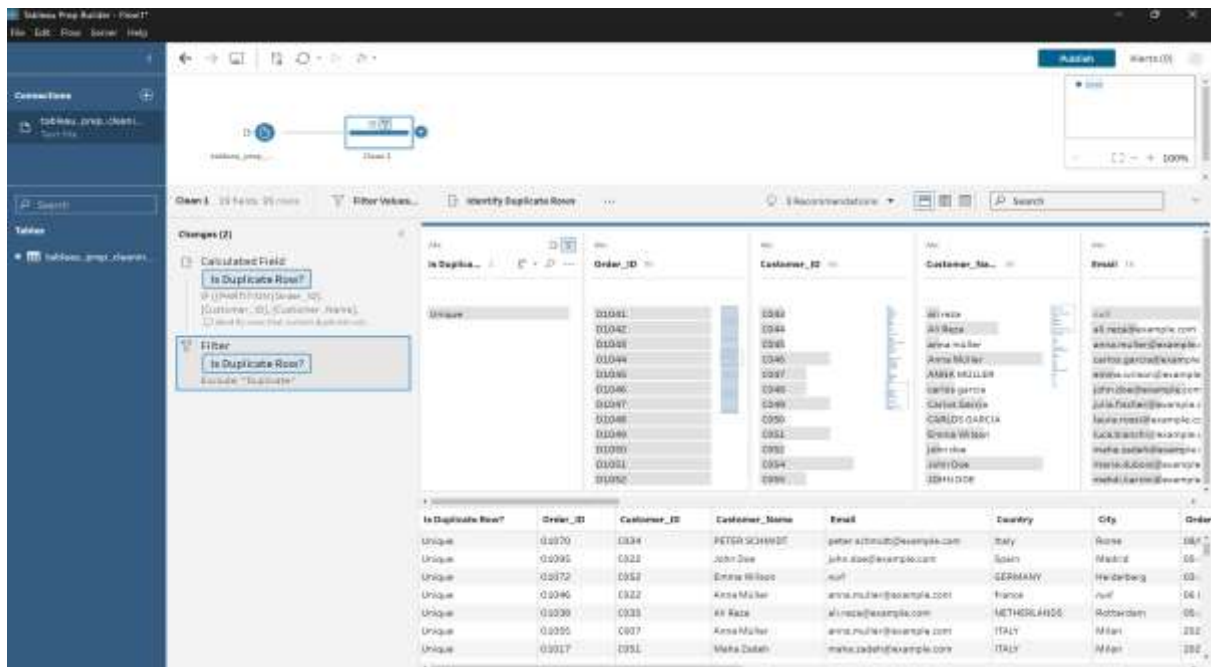
1. Remove duplicate rows → 100 rows to 95
2. Fix Unit_Price (strip € and convert commas)
3. Replace sentinel Unit_Price values (9999.99)
4. Fix Order_Date type → String to Date
5. Fix Ship_Date → Replace "not available" with null, then convert to Date
6. Standardise text columns (Country, Product_Category, Customer_Segment, Payment_Method, Customer_Name)
7. Fill missing values (Satisfaction_Score, Payment_Method, Customer_Segment, Age)
8. Remove zero-quantity rows → 95 to 91
9. Replace Age outlier (130) with median
10. Create calculated fields (Unit_Price_Clean, Calculated_Sales, Age_Group, Order_Year, Order_Month)
11. Apply Min-Max scaling to Unit_Price and Age

6. Cleaning and Transformation

All transformations were executed in Tableau Prep Builder across 14 sequential Clean steps. The dataset was reduced from 100 rows and 18 columns to **91 rows and 32 fields**.

6.1 Duplicate Removal

Five exact duplicate row pairs were removed using the *Identify Duplicate Rows* function. Row count: 100 → 95

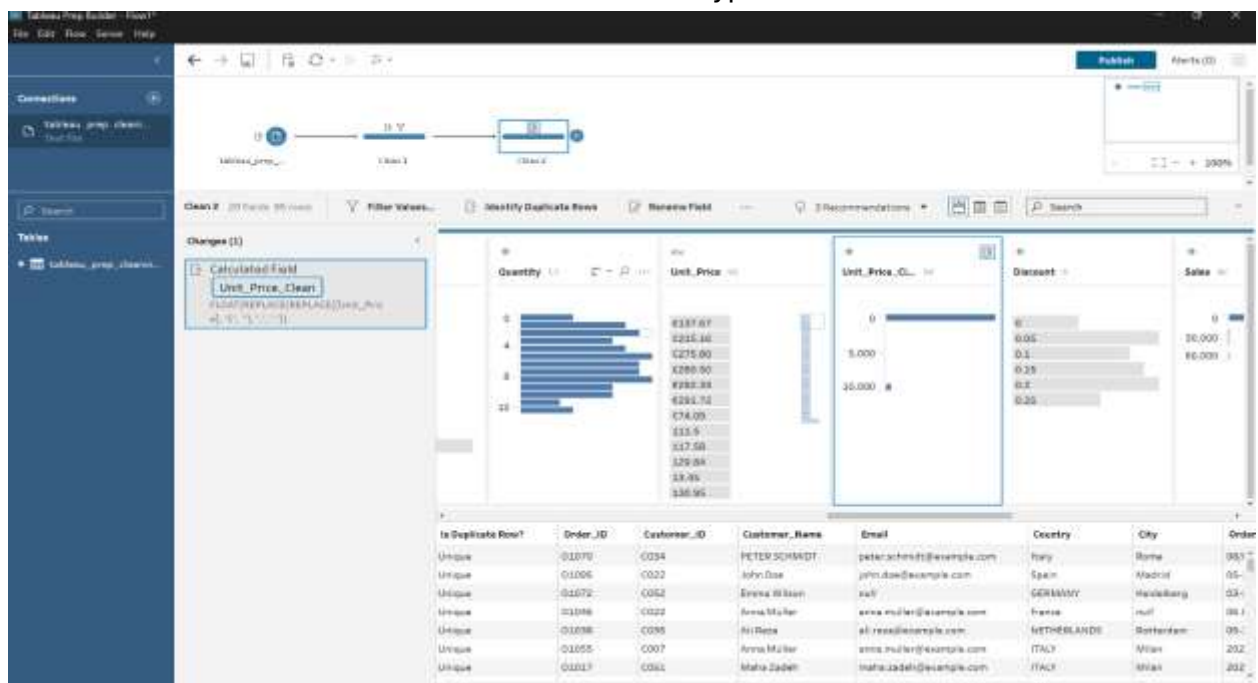


6.2 Unit_Price Type Fix

A calculated field Unit_Price_Clean was created using:

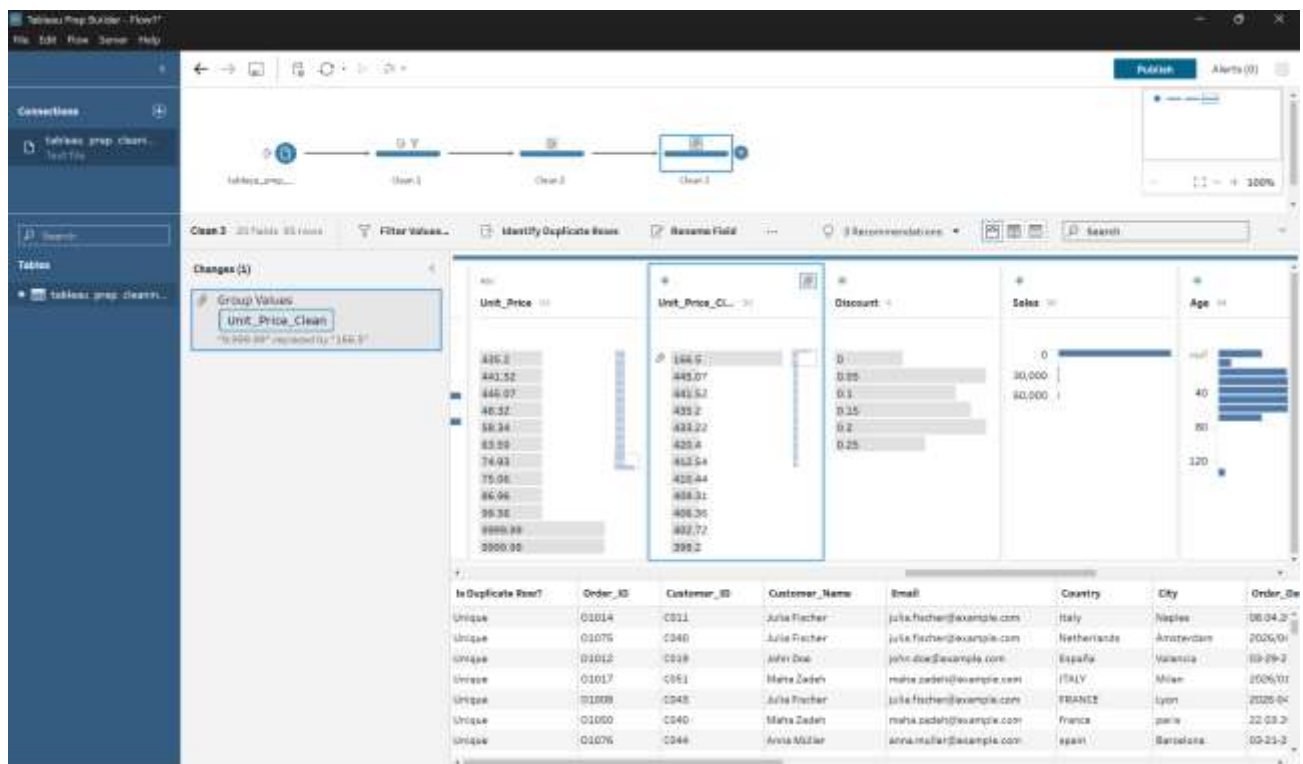
`FLOAT(REPLACE(REPLACE([Unit_Price], '€', ''), ', ', ''))`.

All 13 affected rows (8 with € prefix, 5 with comma decimals) were corrected and the column now holds a numeric float type.



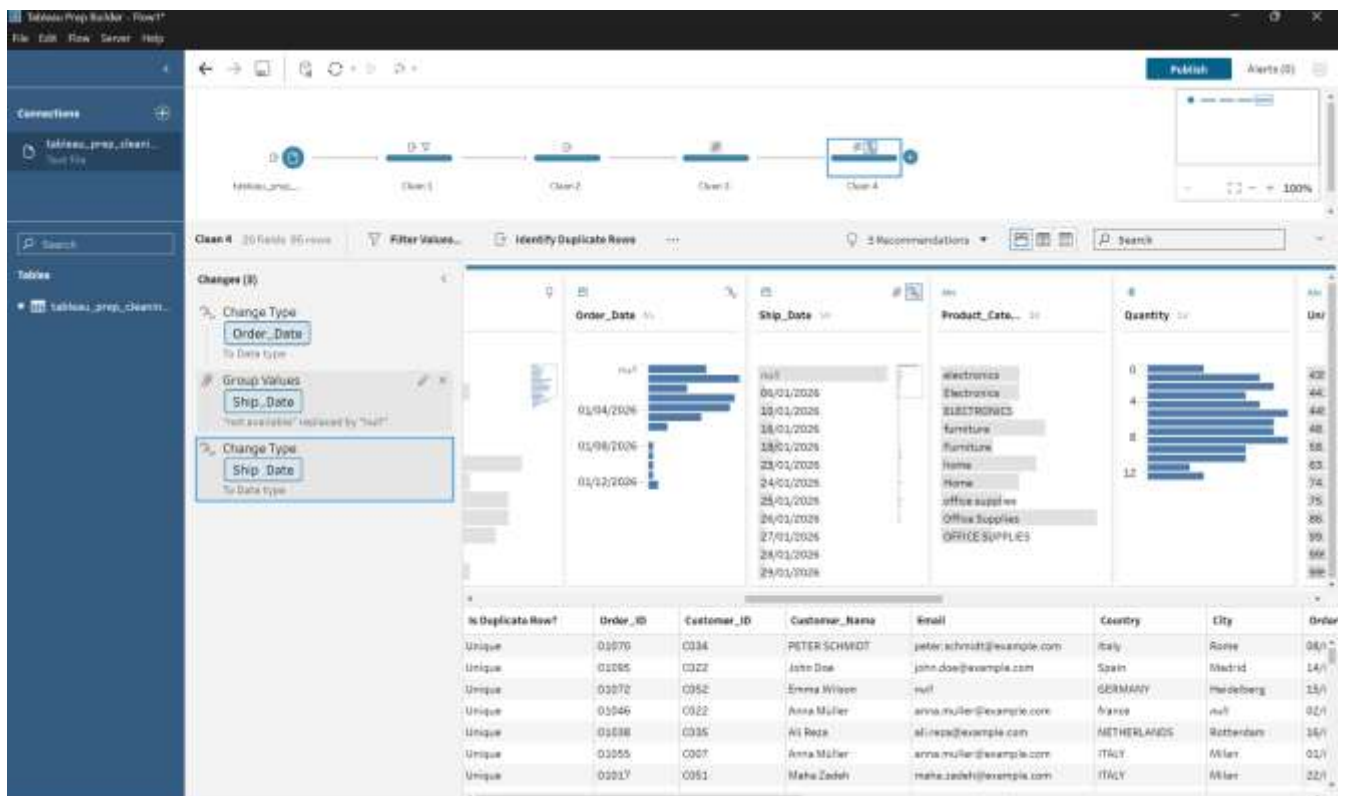
6.3 Sentinel Price Replacement

The placeholder value 9999.99 in Unit_Price_Clean was replaced with the median value **166.50** across 4 affected rows. The maximum real Unit_Price after correction is 445.07



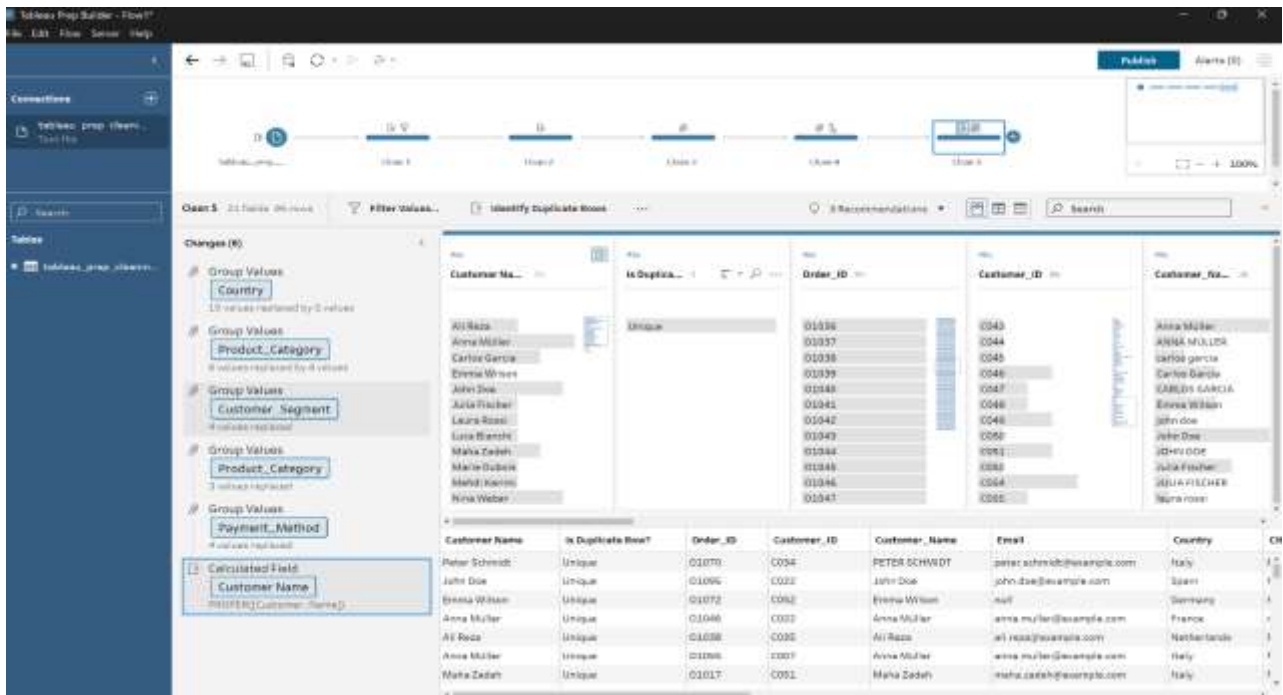
6.4 Date Corrections

Order_Date was converted from String to Date using the Tableau Prep column type selector. Ship_Date had "not available" replaced with null first, then was converted to Date. Both columns now display the calendar icon.



6.5 Text Standardisation

Group and Replace (Pronunciation method) was applied to Country, Product_Category, Customer_Segment, and Payment_Method. Customer_Name was standardised to Title Case using MAKEPROPER([Customer_Name]). Post-cleaning unique counts: Country = 5, Product_Category = 4, Customer_Segment = 4 (including "Unknown"), Payment_Method = 5 (including "Unknown")



6.6 Missing Value Fill

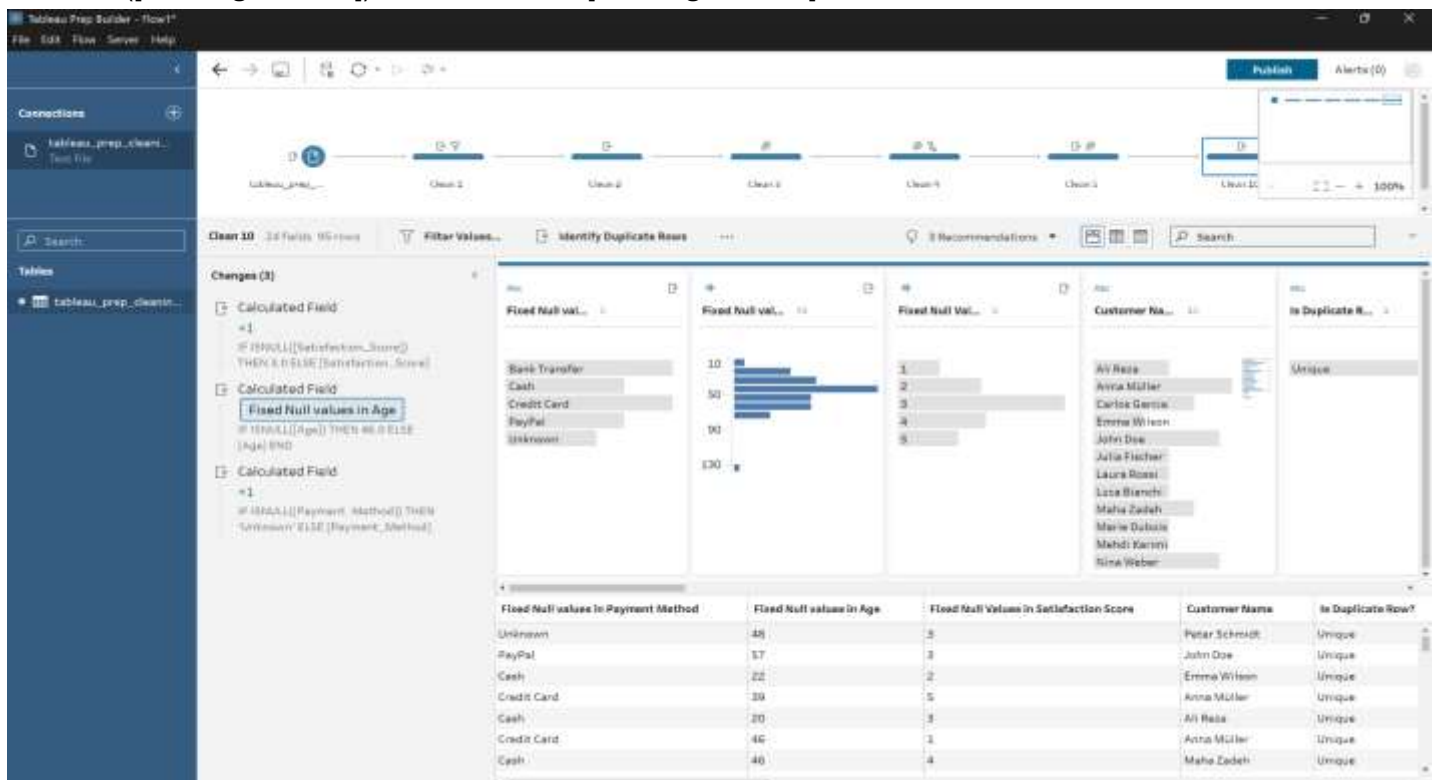
Four critical columns were filled using calculated fields:

IF ISNULL([Satisfaction_Score]) THEN 3.0 ELSE [Satisfaction_Score] END

IF ISNULL([Payment_Method]) THEN 'Unknown' ELSE [Payment_Method] END

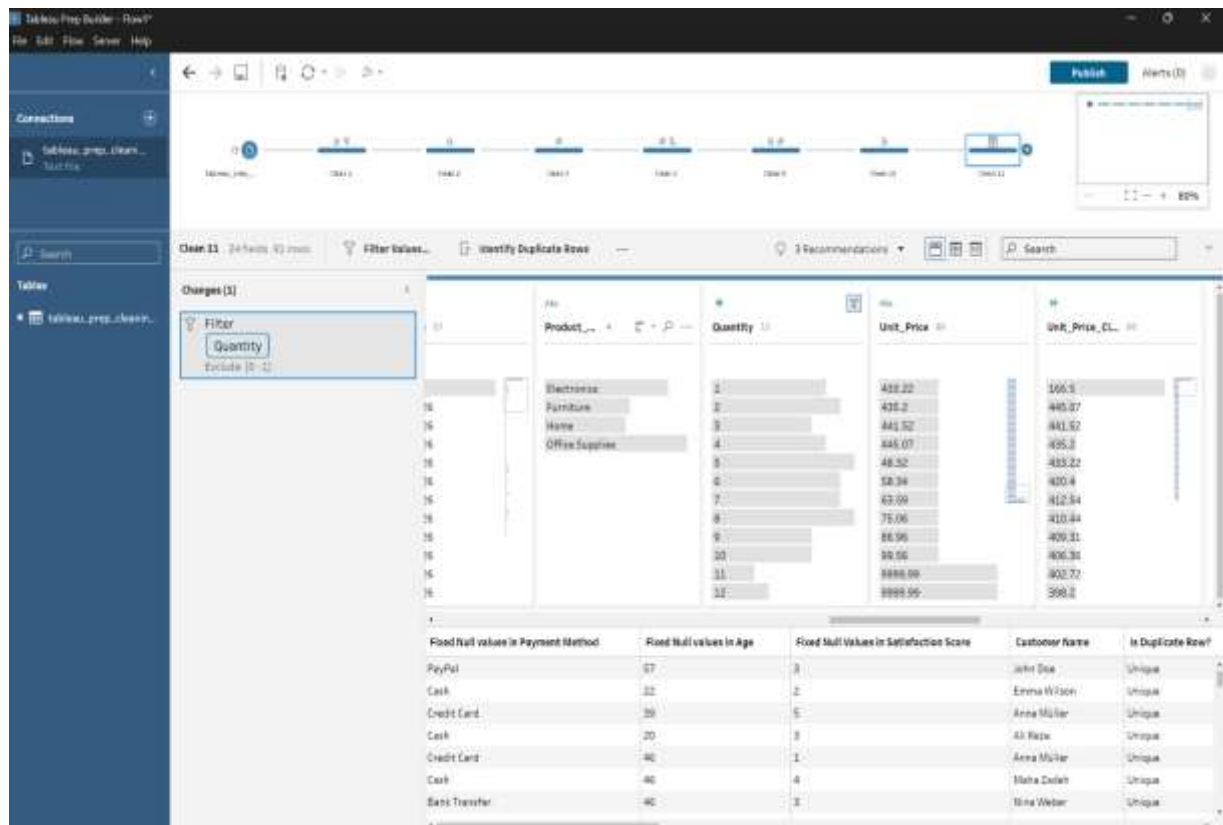
IF ISNULL([Customer_Segment]) THEN 'Unknown' ELSE [Customer_Segment] END

IF ISNULL([Fixed Age Outlier]) THEN 46.0 ELSE [Fixed Age Outlier] END



6.7 Zero-Quantity Row Removal

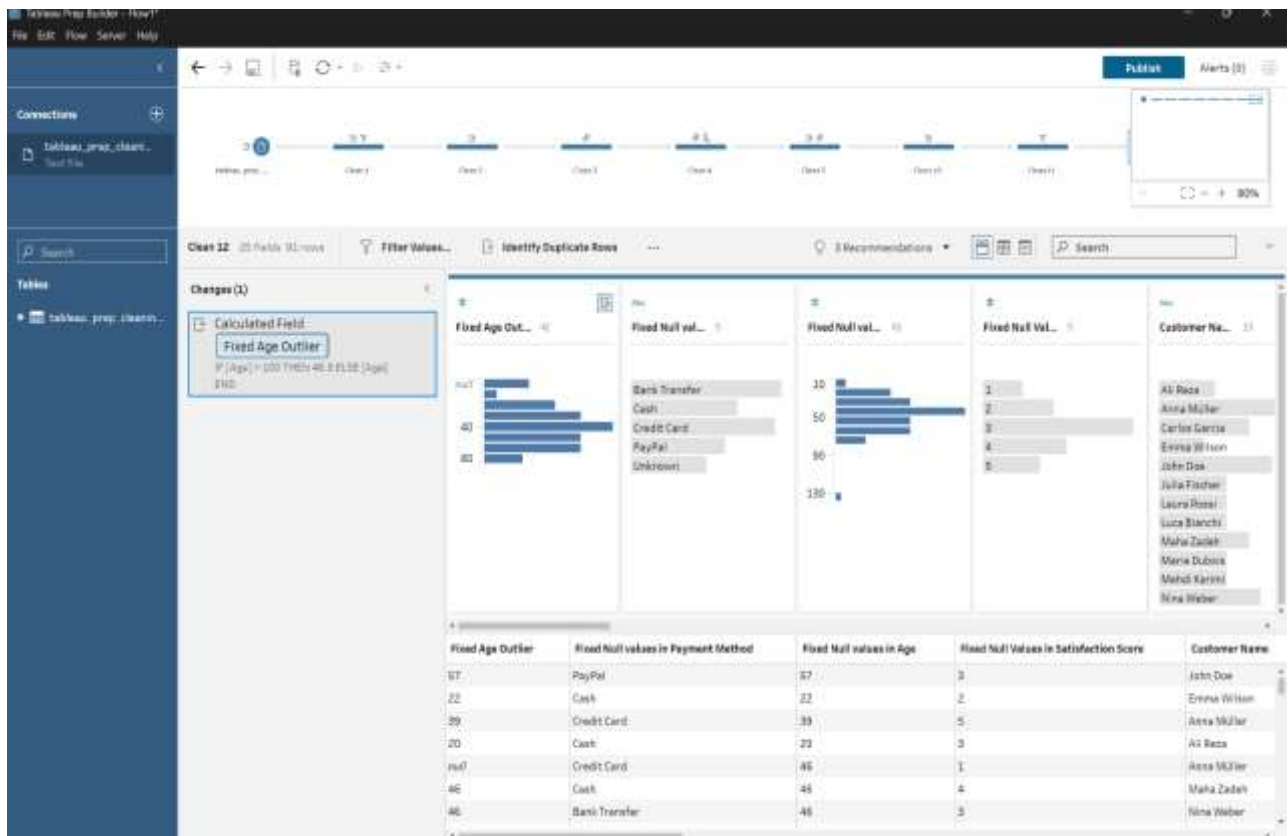
Four rows (O1007, O1035, O1056, O1070) with Quantity = 0 were excluded via filter. Row count: 95 → 91.



6.8 Age Outlier Fix

The impossible value 130 was replaced with the median 46.0 using:

IF [Age] > 100 THEN 46.0 ELSE [Age] END

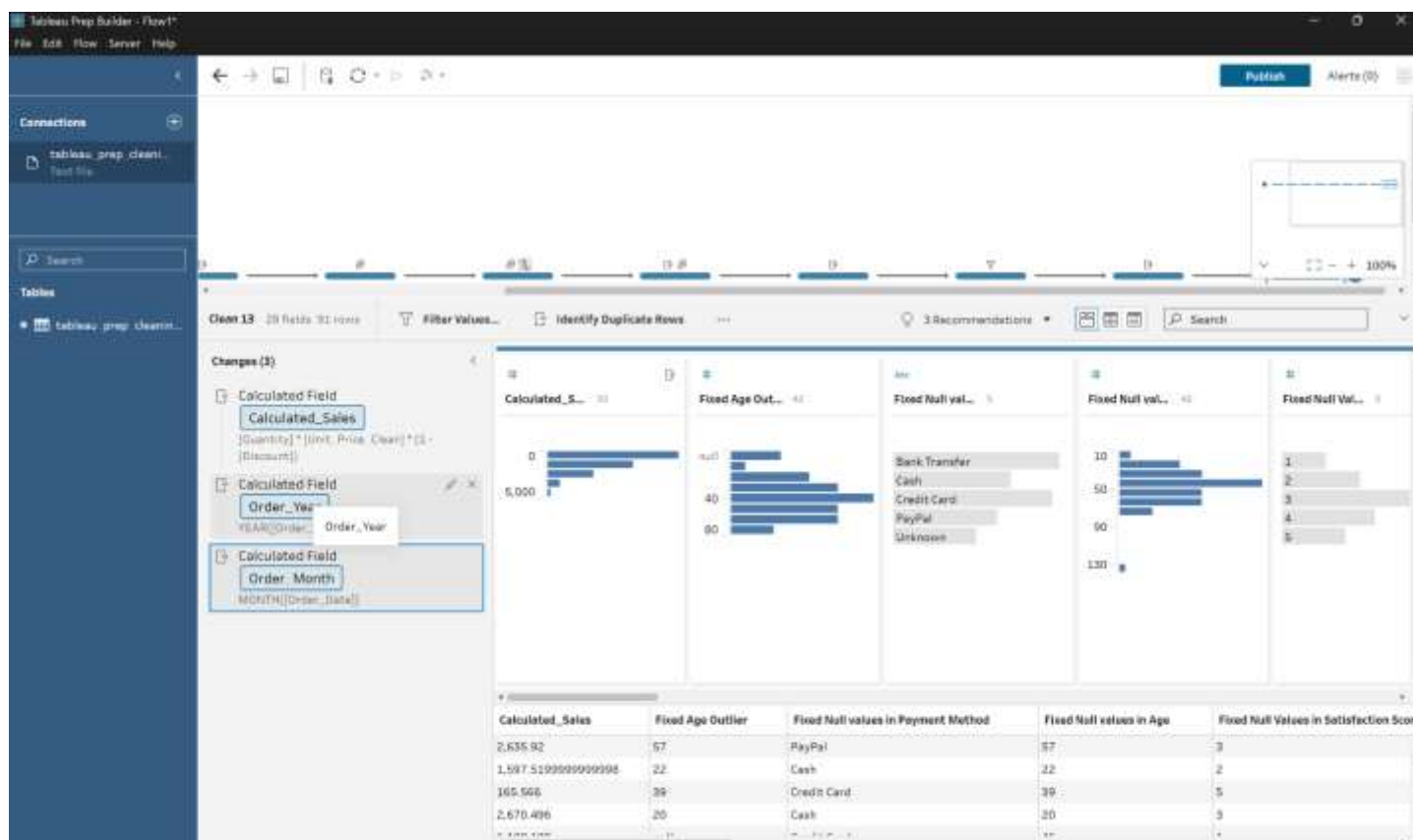


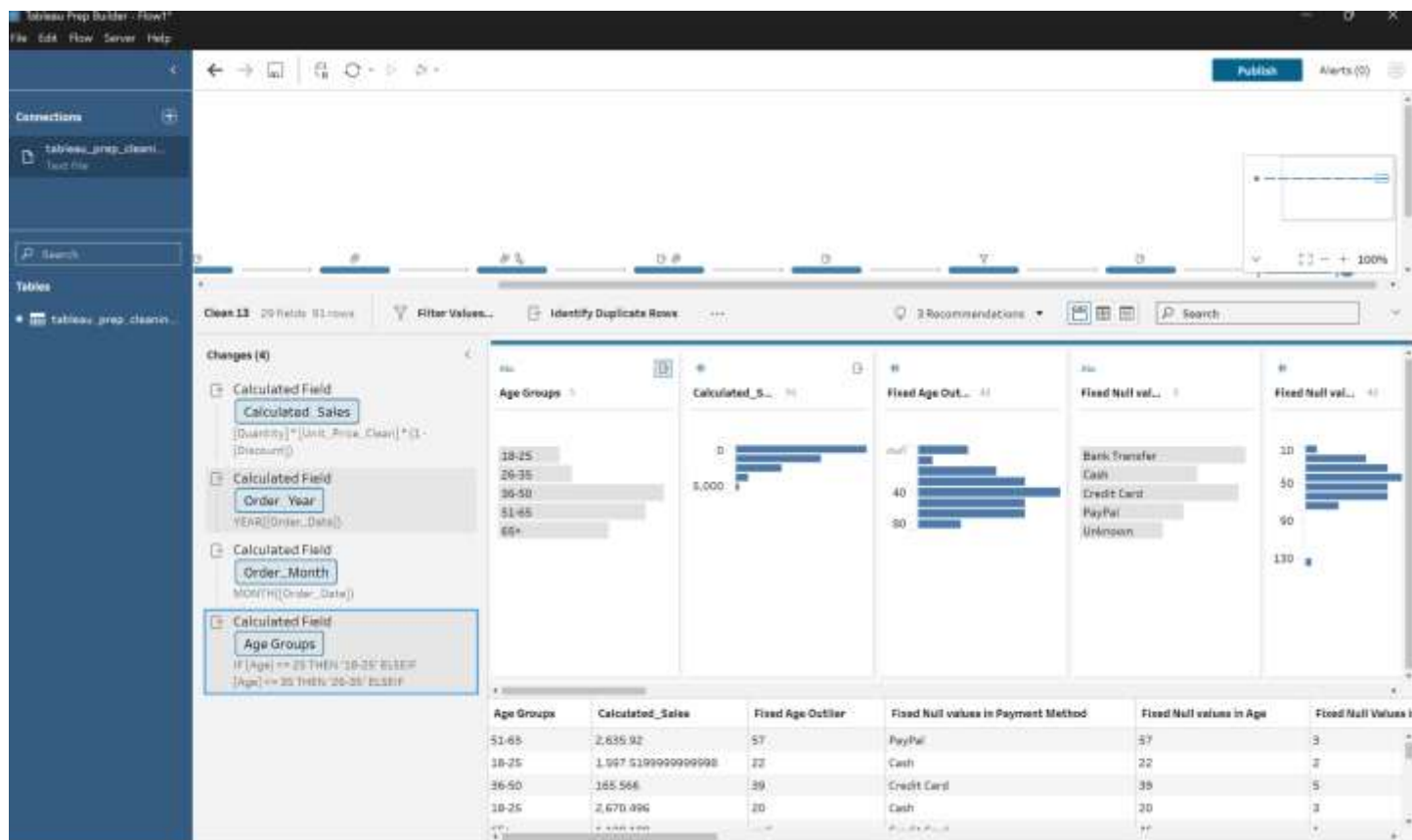
/

6.9 New Calculated Fields

Field Name	Formula	Purpose
Calculated_Sales	$[\text{Quantity}] * [\text{Unit_Price_Clean}] * (1 - [\text{Discount}])$	Verified sales amount
Order_Year	$\text{YEAR}([\text{Order_Date}])$	Year-level analysis
Order_Month	$\text{MONTH}([\text{Order_Date}])$	Monthly trend analysis
Age_Groups	5-bin IF statement (18-25, 26-35, 36-50, 51-65, 65+)	Demographic segmentation
Unit_Price_Scaled	$([\text{Unit_Price_Clean}] - 13.45) / (445.07 - 13.45)$	Min-Max normalisation
Age_Scaled	$([\text{Age_Clean}] - 16.0) / (76.0 - 16.0)$	Min-Max normalisation

Calculated_Sales matched the original Sales column across all 91 rows with zero discrepancies.





7. Validation Report (Post-Cleaning)

7.1 Row Count Comparison

Stage	Row Count	Reason
Original dataset	100	Starting point
After removing duplicates	95	5 exact duplicate pairs removed
After removing zero-quantity rows	91	4 rows with Quantity = 0 excluded
Final cleaned dataset	91	✓ Confirmed

The row count was verified from the toolbar of the Clean 14 step: "32 fields 91 rows".

7.2 Missing Values Re-check

All critical columns reduced to zero nulls:

Column	Nulls Before	Nulls After	Method
Satisfaction_Score	15	0	Filled with median 3.0

Column	Nulls Before	Nulls After	Method
Payment_Method	14	0	Filled with "Unknown"
Customer_Segment	12	0	Filled with "Unknown"
Age (Age_Clean)	7	0	Filled with median 46.0

Remaining nulls in Email (~10), Notes (~10), Order_Date (5), Ship_Date (2), and City (3) are intentional and acceptable.

7.3 Duplicate Re-check

After cleaning, the *Identify Duplicate Rows* function highlighted zero rows. Order_ID now shows 91 unique values across 91 rows, confirming it functions as a valid primary key.

7.4 Data Type Confirmation

All three previously incorrect data types were corrected:

Column	Before	After	Status
Unit_Price	String (Abc)	Float via Unit_Price_Clean	✓
Order_Date	String (Abc)	Date (calendar icon)	✓
Ship_Date	String (Abc)	Date (calendar icon)	✓

All 8 new calculated fields are correctly typed as Float (#), Integer (#), or String (Abc) as expected.

7.5 Key Relationships Validation

- **Order_ID:** 91 unique values, 0 nulls, 0 duplicates — valid primary key
- **Numeric ranges:** Quantity (1–12), Age_Clean (16–76), Unit_Price_Clean (~13–445), Discount (0.00–0.25), Satisfaction_Score (1.0–5.0) — all valid
- **Scaled fields:** Unit_Price_Scaled and Age_Scaled confirmed in range 0.0–1.0
- **Sales formula consistency:** 91/91 rows matched — zero discrepancies

7.6 Overall Validation Summary

Check	Before Cleaning	After Cleaning	Status
Total rows	100	91	✅ 9 rows removed — all explainable
Total columns/fields	18	32	✅ 14 new fields added
Duplicate rows	10	0	✅
Nulls in critical columns	76	0	✅
Wrong data types	3	0	✅
Text inconsistencies	4 columns	0	✅
Age outlier (130)	Present	Replaced with 46.0	✅
Sentinel prices (9999.99)	4 rows	Replaced with median	✅
Order_ID as primary key	Invalid	Valid (91 unique)	✅
Scaled field ranges	Not present	0.0–1.0 verified	✅

8. Export

The cleaned and validated dataset — **91 rows, 32 fields** — was exported from Tableau Prep Builder via the Output step as a .csv file for use in Tableau Desktop or further downstream analysis. The export contains all original columns plus 14 new derived fields including calculated sales, scaled numerics, age groups, and time-based fields.

9. Pipeline Context: ETL vs. ELT

9.1 Where Does Profiling Happen in ETL?

In an ETL (Extract, Transform, Load) pipeline, profiling occurs **before loading** — inside the pipeline tool itself. The raw data is profiled and transformed prior to reaching the destination system. Only clean, validated data is loaded.

text

Source File → [EXTRACT] → [PROFILE HERE] → [TRANSFORM] → [LOAD] → Tableau Desktop

In this assignment, all profiling was performed in the **Clean 1 Profile Pane inside Tableau Prep** before exporting — this is textbook ETL profiling.

9.2 Where Does Profiling Happen in ELT?

In an ELT (Extract, Load, Transform) pipeline, profiling happens **after loading** — the raw data is loaded into the target system first, and profiling and transformation happen inside that system using SQL or tools like dbt.

text

Source File → [EXTRACT] → [LOAD RAW DATA] → [PROFILE HERE] → [TRANSFORM IN PLACE] → Clean Layer

The raw data is always preserved in a raw layer, and new clean layers are built on top of it.

9.3 Which Approach Is Better for This Dataset?

ETL is the correct and better choice for this dataset.

Criterion	ETL	ELT
When transformation happens	Before loading	After loading
Where profiling happens	In pipeline tool (Tableau Prep)	In target system (SQL / warehouse)
Raw data preserved	✗ Only clean data stored	✓ Raw layer always accessible
Schema flexibility	Low — rules defined upfront	High — rules can change anytime
Best for	Structured, small/medium, known schema	Large-scale, cloud-native, exploratory
Scalability	Limited	High
Tool used in this assignment	✓ Tableau Prep	Snowflake / BigQuery / dbt
Correct for this dataset	✓ Yes	✗ No

The reasons ETL is the better fit are:

- The dataset is small (100 rows, 18 columns) — no cloud compute is needed.
- The schema is fully structured and fixed — all cleaning rules were definable before loading.
- The destination is Tableau Desktop, which works best with pre-cleaned data.
- ELT would require loading raw, uncleaned data with mixed date formats, € symbols, and inconsistent casing directly into a warehouse — there is no warehouse in this assignment.
- Raw data preservation (ELT's key advantage) was not required here.

This dataset is small, fully structured, has a fixed schema, and all cleaning rules were fully definable before loading. Profiling was performed correctly in Tableau Prep Builder before export — a complete, correctly executed ETL pipeline.